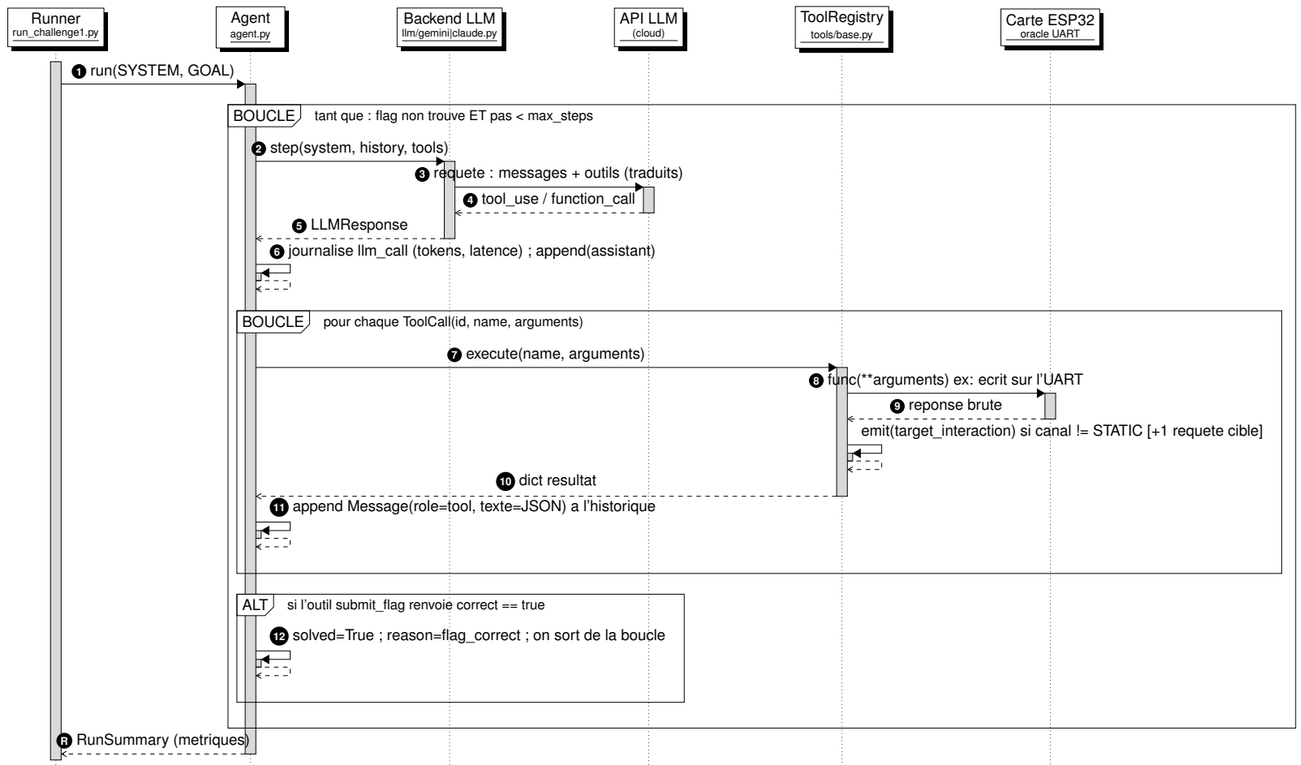
Appel d'outil par l'agent — diagramme de sequence

Harnais ESC 2026 / Serial Gateway — un "pas" de la boucle



Explication etape par etape

1. Le lanceur demarre un run : il passe les *consignes* (SYSTEM) et l'*objectif* (GOAL) a l'agent, et ouvre le port serie vers la carte.
2. A chaque tour de boucle, l'agent appelle `step(system, history, tools)` sur le backend : il fournit tout l'historique et la liste des outils disponibles.
3. Le backend *traduit* l'historique et les outils dans le format de l'API (Gemini / Claude) et envoie la requete.
4. Le modele ne lance rien lui-meme : il *repond* qu'il veut appeler un outil (bloc `tool_use` / `function_call`) avec des arguments.
5. Le backend *normalise* cette reponse en `LLMResponse` (`text`, liste de `ToolCall`, `usage` = tokens).
6. L'agent journalise l'appel LLM (tokens, latence) et ajoute le tour "assistant" a l'historique.
7. Pour chaque `ToolCall`, l'agent appelle `ToolRegistry.execute(name, arguments)`.

8–9. Le registre retrouve la fonction Python de l'outil et l'execute (`func(**arguments)`) : par ex. `uart_send` ecrit la commande sur l'UART et lit la reponse brute de la carte.

10. Le resultat (toujours un `dict`) revient a l'agent. Si le canal n'est pas `STATIC`, l'appel est compte comme *une requete cible* (metrique d'efficacite).
11. L'agent serialise le resultat en **JSON** et l'ajoute comme message `role=tool`. Au pas suivant, le backend le renverra au modele, *reapparie* par `id` (Claude) ou par `nom` (Gemini).
12. Si `submit_flag` valide le flag, le run s'arrete (succes) ; sinon la boucle continue jusqu'au plafond `max_steps`.

R. L'agent renvoie un `RunSummary` (resolu ?, requetes cible, appels LLM, tokens, temps) au lanceur, qui l'affiche.

Types echanges (`llm/base.py`)

`ToolSpec` : nom + description + *schema JSON* des arguments (ce que voit le modele).

`ToolCall` : `id`, `name`, `arguments` (ce que le modele *demande*).

`Message` : un tour d'historique (`user` / `assistant` / `tool`).

`LLMResponse` : `text` + `tool_calls` + `usage`.

`Usage` : tokens (input / output / cache / thinking).

Idee cle

Le LLM decide (ToolCall), *le registre agit* (fonction Python + mesure), *le resultat revient* au modele au pas suivant. La boucle `agent.py` orchestre tout **sans connaitre** ni le fournisseur ni le challenge : on n'ajoute qu'un fichier dans `llm/` (nouveau modele) ou `challenges/` (nouvelle cible).